Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Camera and Media

CAMERA

Introduction

- Mobile cameras are the new definition of communication and entertainment.
- People use the camera to capture still photos or videos to share with friends and family, upload selfies or videos to social network platforms, make video calls, and more.
- almost every app out there uses the camera to perform various tasks.

Introduction

- On Android, if you want to access the device camera you need to create a **SurfaceView** to render previews of what the camera sensor is picking up.
- On iOS, you need to know about the **AVFoundation Capture** that provides a high-level architecture to build a custom camera UI.
- If you are just getting started with mobile app development, then it will be quite difficult to understand both those APIs.

Getting Started

Getting Started

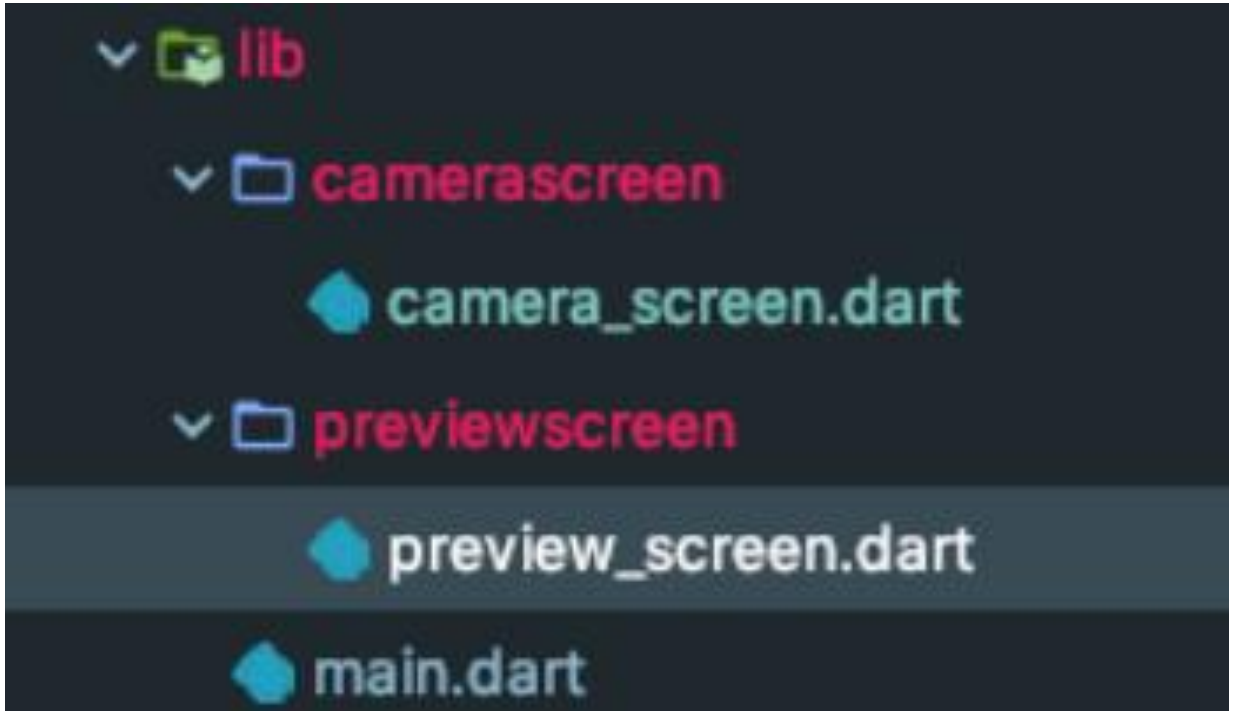
Stage to integrate the camera on the UI:

- **Download Materials** : required libraries and some boilerplate code
- **Open an existing Android Studio project**: Select the starter project which you just downloaded
- **Get dependencies on the Packages get**: pull down the project dependencies
- **Run:** run the app and see the following screen on your mobile phone

Getting Started

Screen on your mobile phone, iOS Simulator, or Android emulator

Exploring the Project



Exploring the Project

Exploring the Project:

- **camera_screen.dart**: the camera preview screen where you can click a picture and toggle between front or back camera
- **preview_screen.dart** : the screen where you will see the preview of the image you clicked and will have the option to share that image with your friend
- **main.dart :** the root widget of your app.
- **pubspec.yaml** :to see all the dependencies required for the app

Exploring the Project

```
name: click_to_share
description: A new Flutter application.
version: 1.0.0+1

environment:
  sdk: ">=2.1.0 <3.0.0"

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^0.1.2
  camera: ^0.5.2+1
  path_provider: ^1.1.0
  path: ^1.6.2
  esys_flutter_share: ^1.0.2
```

all the dependencies required for the app

Exploring the Project

There are four libraries:

- **Camera** :A Flutter plugin for iOS and Android allowing access to the device cameras
- **path_provide** : for finding commonly used locations on the filesystem.
- **Path:**cross-platform path manipulation library for Dart.
- **Esys_flutter_share**: sharing files and text with other applications

Coding the Camera Screen

1. Open the **main.dart** file

```
import 'package:flutter/material.dart';
import 'camerascreen/camera_screen.dart';

class CameraApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CameraScreen(),
    );
  }
}

void main() => runApp(CameraApp());
```

Coding the Camera Screen

2. **CameraApp** : the root widget. This is the entry point for your app. Its responsibility is to launch
3. **CameraScreen** : you will see the camera preview.
4. **Camera_screen.dart file**: a StatefulWidget, because you will be adding the feature to toggle between the front and back camera

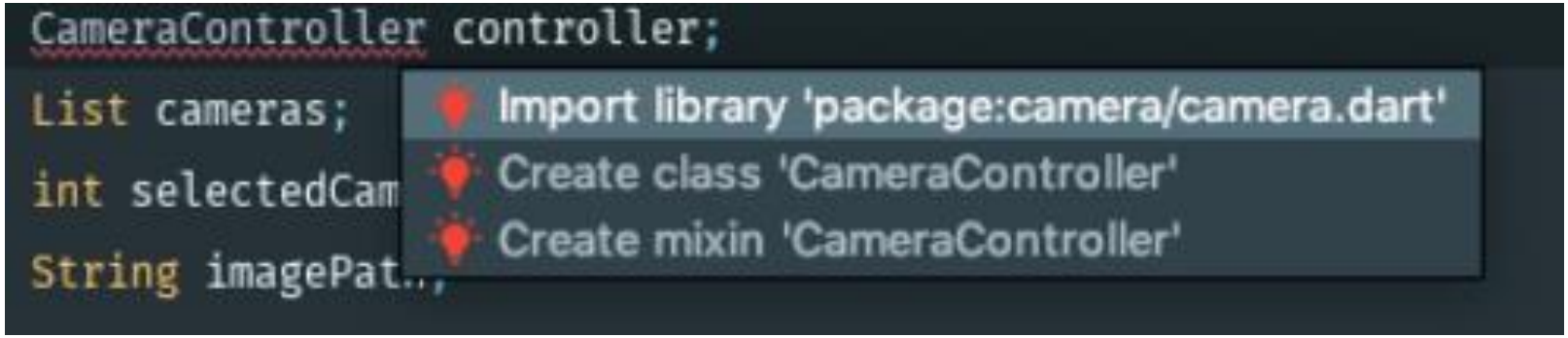
Coding the Camera Screen

5. Going step by step, first add four properties to the **_CameraScreenState** class as shown below:

```
class _CameraScreenState extends State {
  CameraController controller;
  List cameras;
  int selectedCameraIdx;
  String imagePath;
```

Coding the Camera Screen

6. **CameraController**: You might see a red squiggly error line under. If not, that's because the needed import is already in the starter project file. But if so, you need to import the camera package to make the error go away.
7. Click on the line and press **option+return** on macOS6. **Alt+Enter** on PC.
8. **You will see a menu popup**



Coding the Camera Screen

Four properties you just added:

- **CameraController** : for establishing a connection to the device's camera
- **List cameras** :This will hold a list of the cameras available on the device. Normally, the size of the list will be 2 i.e. a front and back camera. The **0** index is for the back camera and the **1** index for the front camera.
- **selectedCameraIdx**: This will hold the current camera index that the user has selected.
- **imagePath** :This will hold the path of the image which you will create using the camera

Displaying the Camera Preview

You've just integrated the camera feature into your app

Displaying the Camera Preview

- you'll need to be on **a physical device**. Run the project and you should see the following screen
- **Integrated** the camera feature into your app
- But as you can see, there is no way to switch between the **front and back camera**, and if you press the capture button nothing happens. You'll implement those features

Switching Cameras

- **_cameraTogglesRowWidget():** showing an icon to the user which indicates the currently selected camera
- **FlatButton**: you can toggle between front and back camera
- **_getCameraLensIcon() method** : returns a specific icon based on the camera selected
- **method _onSwitchCamera()** : toggle between the front and back camera
- .**0** for the back camera and **1** for the front camera
- **CameraDescription**: hold the camera object based on the updated index and then again initialize
- **_initCameraController():** initialize the CameraController
- **Build and run the app to your device:** Tap the icon at the bottom-left corner of the screen to toggle between the front and back camera

Switching Cameras

Stage to Switching Cameras:

1. **_cameraTogglesRowWidget():**showing an icon to the user which indicates the currently selected camera

```
Widget _cameraTogglesRowWidget() {
  if (cameras == null || cameras.isEmpty) {
    return Spacer();
  }

  CameraDescription selectedCamera = cameras[selectedCameraIdx];
  CameraLensDirection lensDirection = selectedCamera.lensDirection;

  return Expanded(
    child: Align(
      alignment: Alignment.centerLeft,
      child: FlatButton.icon(
          onPressed: _onSwitchCamera,
          icon: Icon(_getCameraLensIcon(lensDirection)),
          label: Text(

"${lensDirection.toString().substring(lensDirection.toString().indexOf('.') + 1)}")),
    ),
  );
}
```

Switching Cameras

2. FlatButton: you can toggle between front and back camera
_getCameraLensIcon() method : returns a specific icon based on the camera selected

```
IconData _getCameraLensIcon(CameraLensDirection direction) {
  switch (direction) {
    case CameraLensDirection.back:
      return Icons.camera_rear;
    case CameraLensDirection.front:
      return Icons.camera_front;
    case CameraLensDirection.external:
      return Icons.camera;
    default:
      return Icons.device_unknown;
  }
}
```

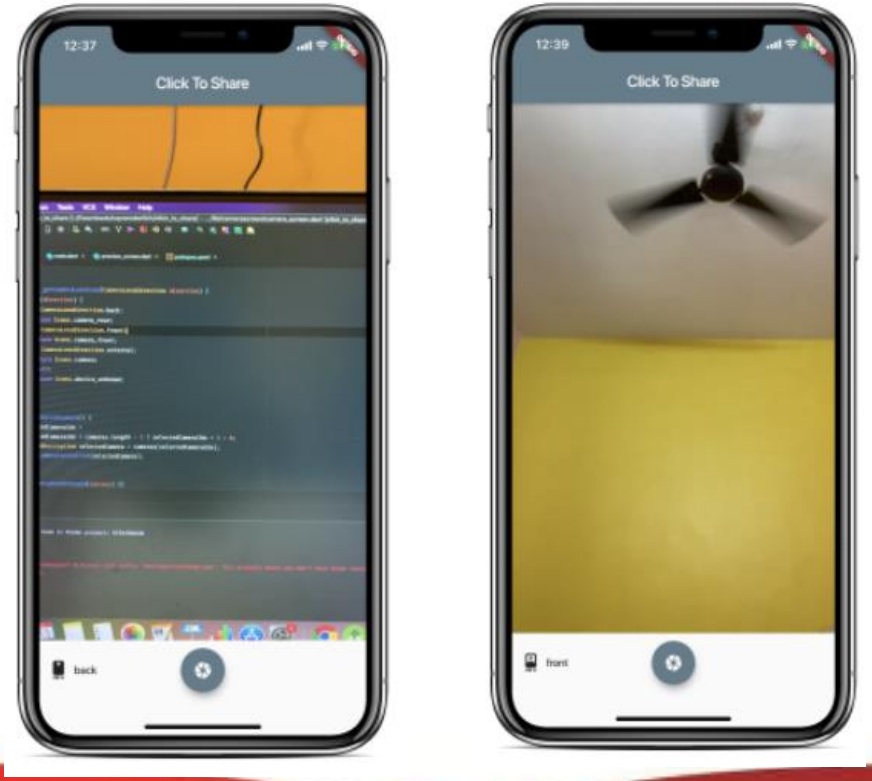
Switching Cameras

3. **method _onSwitchCamera()** : toggle between the front and back camera, **.0** for the back camera and **1** for the front camera **CameraDescription**: hold the camera object based on the updated index and then again initialize**_initCameraController():** initialize the CameraController

```
void _onSwitchCamera() {
  selectedCameraIdx =
  selectedCameraIdx < cameras.length - 1 ? selectedCameraIdx + 1 : 0;
  CameraDescription selectedCamera = cameras[selectedCameraIdx];
  _initCameraController(selectedCamera);
}
```

Switching Cameras

4. Build and run the app to your device: Tap the icon at the bottom-left corner of the screen to toggle between the front and back camera



Taking a Picture

Code step-by-step is as follows:

1. **join(..)** :will take different paths and join them to make a final path. For example: p.join('path', 'to', 'foo'); // -> 'path/to/foo’
2. **takePicture(path)** : will capture a raw image and write it to the path provided as an argument.
3. **PreviewImageScreen**: to preview the image and share it with your friends.

Taking a Picture

Sharing the Image

Stage to Sharing the Image:

1. click on the **Share Button**: you will be presented with a **Bottomsheet**
2. **Bottomsheet:** you can share the image you have just clicked
3. Open the **preview_screen.dart:** to see the code behind the share button.
4. The **getBytesFromFile()** method : converts the image file into ByteData

```
Future<ByteData> getBytesFromFile() async {
  Uint8List bytes = File(widget.imagePath).readAsBytesSync() as Uint8List;
  return ByteData.view(bytes.buffer);
}
```

Sharing the Image

5. The **onPressed** handler for the share button calls **getBytesFromFile().**

```
onPressed: () {
  getBytesFromFile().then((bytes) {
    Share.file('Share via:', basename(widget.imagePath),
        bytes.buffer.asUint8List(), 'image/png');
  });
},
```

6. After that call returns, the bytes from the file are passed to the file() **method of Share** class. The Share class is a part of **esys_flutter_share** plugin which you already added in the starter project

Sharing the Image

You've successfully integrated the camera, and it's time to play around with the app you just created. Take some amazing pictures and share them with your friends

Where to Go From Here?

MEDIA

- **iOS** :If you need to access videos using http (rather than https) URLs, you will need to add the appropriate NSAppTransportSecurity permissions to your app's *Info.plist* file, located in **<project root>/ios/Runner/Info.plist.**
- **Android** : If you are using network-based videos, ensure that the following permission is present in your Android Manifest file, located in **<project root>/android/app/src/main/AndroidManifest.xml:**

Supported Formats

- **On iOS,** the backing player is AVPlayer. The supported formats vary depending on the version of iOS, AVURLAsset class
  has audiovisualTypes that you can query for supported av formats.
- **On Android**, the backing player is ExoPlayer
- **On Web**, available formats depend on your users' browsers (vendor and version).

To use the video_player package to stream a video from the internet with basic play and pause controls using the following steps:

1. **Add the video_player dependency.:** This recipe depends on one Flutter plugin: video_player. First, add this dependency to your pubspec.yaml
2. **Add permissions to your app**: update your android and ios configurations to ensure that your app has the correct permissions to stream videos from the internet.
3. **Create and initialize a VideoPlayerController**.:plugin installed with the correct permissions, create a VideoPlayerController. The VideoPlayerController class allows you to connect to different types of videos and control playback
4. **Display the video player:** plugin provides the VideoPlayer widget to display the video initialized by the VideoPlayerControlle
5. **Play and pause the video**.:To begin playback, call the play() method provided by the VideoPlayerController. To pause playback, call the pause() method

To create and initialize the VideoPlayerController do the following:

1. **Create a StatefulWidget** with a companion State class
2. **Add a variable to the State class** to store the VideoPlayerController
3. **Add a variable to the State class to store the Future returned** from VideoPlayerController.initialize
4. **Create and initialize the** controller in the initState method
5. **Dispose of the controller** in the dispose method

The plugin is using most of the code of Google official package video_player and has some extra features such as background playing, can have a playlist as a data source.

Some features of the plugin:

- Ability to play Audio/Video URL
- Persistent Notification
- Background Playback
- Playlist Support
- Ability to provide the source at runtime (you will able to change the playing content without re-instantiate it)
- Able to navigate through the playlist

Stage to create a media player application from **assets** as well as from the **network** using flutter:

1. Let's begin the task by creating a new workspace for application. use below command to create. I named my app as **media.**

```
(base) Pasupuletis-MacBook-Air:projects pasupuletirajeshbabu$ flutter create media
Creating project media...
  media/ios/Runner.xcworkspace/contents.xcworkspacedata (created)
  media/ios/Runner.xcworkspace/xcshareddata/IDEWorkspaceChecks.plist (created)
  media/ios/Runner.xcworkspace/xcshareddata/WorkspaceSettings.xcsettings (created)
  media/ios/Runner/Info.plist (created)
  media/ios/Runner/Assets.xcassets/LaunchImage.imageset/LaunchImage@2x.png (created)
  media/ios/Runner/Assets.xcassets/LaunchImage.imageset/LaunchImage@3x.png (created)
  media/ios/Runner/Assets.xcassets/LaunchImage.imageset/README.md (created)
  media/ios/Runner/Assets.xcassets/LaunchImage.imageset/Contents.json (created)
  media/ios/Runner/Assets.xcassets/LaunchImage.imageset/LaunchImage.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-76x76@2x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-29x29@1x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-40x40@1x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-20x20@1x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-1024x1024@1x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-83.5x83.5@2x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-20x20@3x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Contents.json (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-20x20@2x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-29x29@3x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-40x40@2x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-60x60@3x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-60x60@2x.png (created)
  media/ios/Runner/Assets.xcassets/AppIcon.appiconset/Icon-App-76x76@1x.png (created)
```

2. After creating now we have to install some plugins in pubspec.yaml file. Add these plugins in our pubspec.yaml file
3. This is the main code of your flutter app it will run and create our whole app
   This main page calls the page1() & page2()

main.dart

pubspec.yaml main.dart

```
1  import 'package:flutter/material.dart';
2  import 'Page1/page1.dart';
3  import 'Audio_Page/page2.dart';
4  import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';
5
6  void main() {
7    FlutterStatusbarcolor.setStatusBarColor(Colors.pink);
8    runApp(MyApp());
9  }
10
11 class MyApp extends StatelessWidget {
12   @override
13   Widget build(BuildContext context) {
14     return MaterialApp(
15       initialRoute: '/',
16       routes: {
17         '/Page1': (context) => Page1(),
18         '/Page2': (context) => Page2(),
19       },
20       debugShowCheckedModeBanner: false,
21       home: Page1(),
22     ); // MaterialApp
23   }
24 }
25
```

4. next, we have to create UI pages I will give source code in the below link. This is the first page of the application it shows music and video buttons. we have to select either Music or Video. if we press the Music button it will navigate to Page2.

[page1.dart]

```
page1.dart   pubspec.yaml   main.dart
import 'package:flutter/material.dart';
import 'package:media/Video_page/vieda5.dart';
import 'package:media/core/const.dart';
import '../Audio_Page/page2.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';

class Page1 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    FlutterStatusbarcolor.setStatusBarColor(AppColors.lightBlueShadow);
    return Scaffold(
      backgroundColor: Color(0XFFe5eefc),//maincolor
      appBar: AppBar(
        title: Text('MEDIA PLAYER',
            style: TextStyle(color: Color(0XFF6f7e96),//stylecolor
            ) // TextStyle
        ), // Text
        backgroundColor: Color(0XFFe5eefc),//maincolor
        centerTitle: true,
      ), // AppBar
      body:
      Container(

        color: Color(0XFFe5eefc),//maincolor

        child: Center(
          child: Column(

            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Container(
                margin: const EdgeInsets.all(50.0),

                decoration: BoxDecoration(
                  image: DecorationImage(
                    image: NetworkImage(
                        'https://miro.medium.com/max/1000/0*_KI5ko3_H_L7h2Mj.jpg
                    fit: BoxFit.cover,
```

5. After done the above step we have to select offline or Online. if we click on the offline it will navigate to page3.dart mention in step6 or if we click on the online it will navigate to page4.dart mention in step7

page2.dart

main.dart page2.dart

```dart
import 'package:flutter/material.dart';
import 'package:media/Audio_Page/leo.dart';
import 'package:media/Audio_Page/page3.dart';
import 'package:media/core/const.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';

class Page2 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    FlutterStatusbarcolor.setStatusBarColor(AppColors.lightBlueShadow);
    return Scaffold(
      backgroundColor: Color(0XFFe5eefc),//maincolor
      appBar: AppBar(
        title: Text('MEDIA PLAYER',
            style: TextStyle(color: Color(0XFF6f7e96),//stylecolor
            ) // TextStyle
        ), // Text
        backgroundColor: Color(0XFFe5eefc),//maincolor
        centerTitle: true,
      ), // AppBar
      body:
      Container(

        color: Color(0XFFe5eefc),//maincolor

        child: Center(
          child: Column(

            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Container(
                margin: const EdgeInsets.all(50.0),

                decoration: BoxDecoration(
                  image: DecorationImage(
                    image: NetworkImage(
```

6. Here we can play the offline song. here we have play button, pause button and stop button it takes the song from assets file [page3.dart]

```
main.dart   page2.dart   page3.dart
import 'package:audioplayers/audio_cache.dart';
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';
import 'package:media/core/const.dart';

typedef void OnError(Exception exception);

class LocalAudio extends StatefulWidget {
  @override
  _LocalAudio createState() => _LocalAudio();
}

class _LocalAudio extends State<LocalAudio> {
  Duration _duration = new Duration();
  Duration _position = new Duration();
  AudioPlayer advancedPlayer;
  AudioCache audioCache;

  @override
  void initState() {
    super.initState();
    initPlayer();
  }

  BoxDecoration mybox() {
    return BoxDecoration(border: Border.all(width: 2, color: Colors.red[500]));
  }

  void initPlayer() {
    advancedPlayer = new AudioPlayer();
    audioCache = new AudioCache(fixedPlayer: advancedPlayer);

    advancedPlayer.durationHandler = (d) => setState(() {
      _duration = d;
    });

    advancedPlayer.positionHandler = (p) => setState(() {
```

7. Here we can play the Online song. here we have a play button, pause button, and stop button it takes the song from the online player. I uploaded in AWS cloud
[page4.dart]

```
import 'package:audioplayers/audio_cache.dart';
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';
import 'package:media/core/const.dart';

typedef void OnError(Exception exception);

class OnlineAudio extends StatefulWidget {
  @override
  _LocalAudio createState() => _LocalAudio();
}

class _LocalAudio extends State<OnlineAudio> {
  Duration _duration = new Duration();
  Duration _position = new Duration();
  AudioPlayer advancedPlayer;
  AudioCache audioCache;

  @override
  void initState() {
    super.initState();
    initPlayer();
  }

  BoxDecoration mybox() {
    return BoxDecoration(border: Border.all(width: 2, color: Colors.red[500]));
  }

  void initPlayer() {
    advancedPlayer = new AudioPlayer();
    audioCache = new AudioCache(fixedPlayer: advancedPlayer);

    advancedPlayer.durationHandler = (d) => setState(() {
      _duration = d;
    });

    advancedPlayer.positionHandler = (p) => setState(() {
      _position = p;
```

8. we have to select offline or Online. if we click on the offline it will navigate to page6.dart mention in step9 or if we click on the online it will navigate to page7.dart mention in step10

[page5.dart]

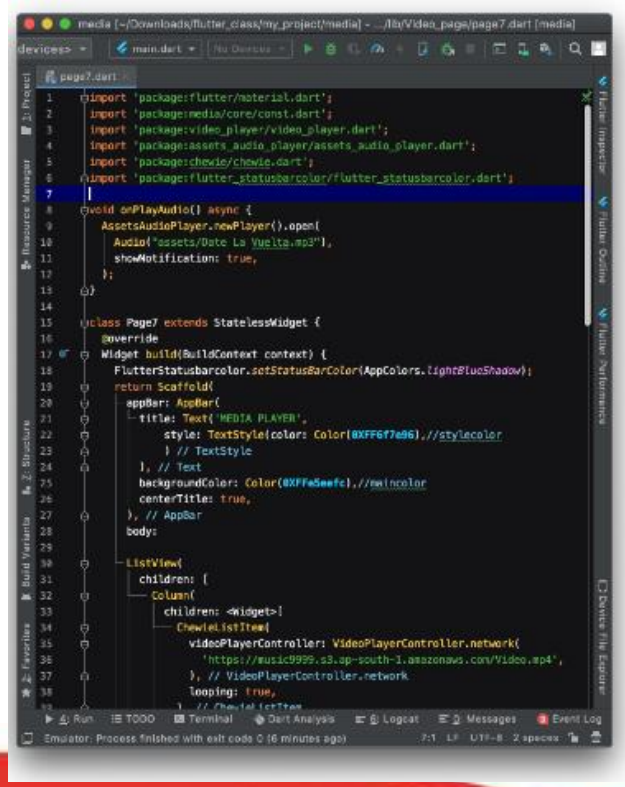
```dart
import 'package:flutter/material.dart';
import 'package:media/Video_page/page6.dart';
import 'package:media/Video_page/page7.dart';
import 'package:media/core/const.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';

class viedo2 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    FlutterStatusbarcolor.setStatusBarColor(AppColors.lightBlueShadow);
    return Scaffold(
      backgroundColor: Color(0XFFe5eefc),//maincolor
      appBar: AppBar(
        title: Text('MEDIA PLAYER',
            style: TextStyle(color: Color(0XFF6f7e96),//stylecolor
            ) // TextStyle
        ), // Text
        backgroundColor: Color(0XFFe5eefc),//maincolor
        centerTitle: true,
      ), // AppBar
      body:
      Container(

        color: Color(0XFFe5eefc),//maincolor

        child: Center(
          child: Column(

            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Container(
                margin: const EdgeInsets.all(50.0),

                decoration: BoxDecoration(
                  image: DecorationImage(
                    image: NetworkImage(
                        'https://miro.medium.com/max/1000/0*_KISko3_H_L7hZMj.jpg
                    fit: BoxFit.cover,
                  ), // DecorationImage
```

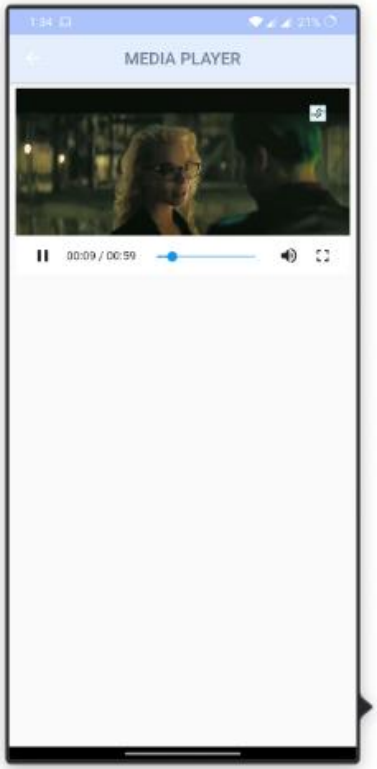
9. After clicking on the offline button.Here we can play offline video. here we have a play button, mute button, and full-screen option.it takes video from assets.

[page6.dart]

```
import 'package:flutter/material.dart';
import 'package:media/core/const.dart';
import 'package:video_player/video_player.dart';
import 'package:chewie/chewie.dart';
import 'package:flutter_statusbarcolor/flutter_statusbarcolor.dart';

class Page6 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    FlutterStatusbarcolor.setStatusBarColor(AppColors.lightBlueShadow);
    return Scaffold(
      appBar: AppBar(
        title: Text('MEDIA PLAYER',
            style: TextStyle(color: Color(0XFF6f7e96),//stylecolor
            ) // TextStyle
        ), // Text
        backgroundColor: Color(0XFFe5eefc),//maincolor
        centerTitle: true,
      ), // AppBar
      body: Container(
        color: AppColors.mainColor,

        child: ListView(
          children: [

            Column(
              children: <Widget>[
                ChewieListItem(
                  videoPlayerController: VideoPlayerController.asset('assets/Vide
                  ), // VideoPlayerController.asset

                  looping: true,
                ), // ChewieListItem
              ], // <Widget>[]
            ), // Column

          ],
        ), // ListView
      ), // Container
```

10. Here we can play Online video. here we have a play button, mute button, and full-screen option. I uploaded the mp4 video in the AWS cloud. it will take from AWS





Any question?

References

- https://flutter.dev
- https://docs.flutter.dev/cookbook/plugins/picture-using-camera
- https://docs.flutter.dev/cookbook/plugins/play-video

Fakultas Informatika
School of Computing
Telkom University

THANK YOU